

DarkSage

Testing & QA

Version 0.5.0 — Draft

Owner: TheSinnerMan

Classification: Internal

Wisdom Over Noise

Source repository: `LightSwornPVP/DarkSage`

Source baseline commit: `8b5cf5bd0341d5d6f70c67302c8bce2069d0929a`

Generation date: `2026-07-25T16:51:31-04:00`

Document ID: `DS-009`

Document Control

Document ID	DS-009
Title	Testing & QA
Version	0.5.0
Status	Draft
Owner	TheSinnerMan
Classification	Internal
Repository	LightSwornPVP/DarkSage
Created	2026-07-24
Last Updated	2026-07-25

Status lifecycle: Draft → Under Review → Approved → Superseded/Deprecated.

Revision History

Version	Date	Author	Summary
0.5.0	2026-07-25	TheSinnerMan / Keeper	Independent-audit blocker-repair (Blockers 2, 3): added DS-QA-027 (Trade Intelligence Package Security Testing, covering DS-SCA-029) and DS-QA-028 (Journal Retention and Deletion Testing, covering DS-JRN-007/DS-API-JRN-004/DS-SCA-028's extension). No prior requirement's content changed.
0.4.0	2026-07-25	TheSinnerMan / Keeper	Independent-audit repair (H1, H3): added DS-QA-024 (Research Intelligence Testing), DS-QA-025 (Journal & Review Testing), and DS-QA-026 (Trade Intelligence Package Testing), closing the H1 test-traceability gap. Renumbered the Founder Vision Completion section from the misnumbered "## 25." (skipping §§16–24) to the correct next-available "## 16." No prior requirement's content changed.
0.3.0	2026-07-25	TheSinnerMan / Keeper	Founder Vision Completion amendment and cross-volume traceability, including Discord notification boundaries where applicable.
0.1.0	2026-07-24	TheSinnerMan	First controlled draft, authored as part of the Batch 1 grouped pass (DS-007/DS-008/DS-009). Consolidates the testing obligations already stated per-requirement across DS-002 through DS-008 into a single testing/QA architecture under the DS-QA prefix — release gates, traceability, defect severity, and category-specific test requirements (unit, integration, contract, regression, e2e, security, performance, data-quality, model evaluation, backtesting validation, deterministic financial calculation).
0.2.0	2026-07-24	TheSinnerMan	Targeted repair pass. DS-009-H1 : repaired DS-QA-009, which incorrectly listed the Planned DS-API-EXE-006 alongside Committed governance boundaries; split into an always-required Committed pipeline-integrity constraint (DS-EXE-001/DS-API-EXE-001 only) and a conditional Planned Trade Proposal submission test (DS-API-EXE-005/006), with an explicit acceptance criterion barring any promotion of DS-API-EXE-006 to Committed. Cross-volume alignment : added DS-QA-021 (Interface State Lifecycle Testing, covering DS-007's new DS-UX-016), DS-QA-022 (Audit-Log Integrity Testing, covering DS-008's new DS-SCA-025), and DS-QA-023 (Incident-Response Lifecycle Testing, covering DS-008's new DS-SCA-026); extended DS-QA-013's acceptance criteria to require scenario-to-test mapping against DS-SCA-024's new threat-scenario table. Corrected stale DS-UX-016/DS-UX-020 references (now DS-UX-017/DS-UX-021) following DS-007's H1 renumbering.

Version	Date	Author	Summary
0.2.1	2026-07-24	TheSinnerMan	Narrow repair pass for the two remaining High findings. DS-007-H1 alignment: DS-QA-021 split into Committed core-lifecycle testing (DS-UX-016) and Planned, conditional assistive-technology announcement testing (DS-UX-022) — the latter is never a Committed release gate before DS-UX-022 is implemented. DS-008-H2 alignment: DS-QA-022 expanded to cover DS-008's new DS-SCA-027 integrity-verification contract — direct-storage-corruption adversarial tests for modification, reordering, truncation, selective deletion, and unauthorized insertion; verification-trigger tests (startup, pre-review, scheduled, post-fault); a fail-closed-on-violation test; an evidence-preservation test; and an anti-silent-rebuild test — going beyond the prior API-level-only tamper tests. §13 Risks and Constraints updated to record DS-QA-021's split.
0.2.2	2026-07-24	TheSinnerMan	Consolidated cleanup pass: renamed DS-QA-001 from "Unit Testing Baseline" to "Requirement Test-Coverage Completeness Baseline" — its actual normative content (every Committed/MVP requirement across DS-002 through DS-009 has at least one recorded automated or documented manual test) was never specific to unit testing as a test type, and no dedicated unit-testing requirement existed elsewhere in this document; the underlying requirement itself is unchanged. Added an explicit Severity Precedence Rule to DS-QA-019: when a defect matches more than one severity level's criteria, the highest-matching severity governs (Critical > High > Medium > Low), stated deterministically so a Critical/High finding touching correctness, security, safety, privacy, reproducibility, or architecture can never be reclassified downward by also matching a lower-severity criterion. No existing blocker category was downgraded; the four severity definitions and their blocking behavior are unchanged.

Version	Date	Author	Summary
0.2.3	2026-07-25	TheSinnerMan	Suite-audit blocker repair (SUITE-H01): DS-QA-019 cited <code>.ai-workflow/AGENT_PROTOCOL.md</code> as Governing Source and <code>.ai-workflow/BLOCKERS.md</code> as the source of its severity vocabulary, violating DS-DEV-025's rule that no local-only workflow file is ever cited as Governing Source by a Committed/MVP requirement. Governing Source now reads DS-009 (normative), DS-DEV-009/DS-DEV-025 (DS-010), DS-ADR-010 (DS-012) — all committed public authority. The four severity definitions, the Severity Precedence Rule, and the release-blocking Acceptance Criteria are restated as fully self-contained and normative on DS-009's own authority, reproducible from the committed repository alone. <code>.ai-workflow/BLOCKERS.md</code> is now explicitly described as historical/operational context only, never required reading to define or apply the severity model. §12 Dependencies' <code>.ai-workflow/AGENT_PROTOCOL.md</code> entry removed and replaced with the DS-010/DS-012 citations above. No severity definition, precedence order, or blocking behavior was weakened or downgraded.

Table of Contents

Document Control	2
Revision History	3
Table of Contents	6
1. Purpose	8
2. Scope	8
3. Audience	8
4. Definitions	8
5. Test Category Definitions	9
DS-QA-001 — Requirement Test-Coverage Completeness Baseline	9
DS-QA-002 — Integration Testing	9
DS-QA-003 — API Contract Testing	9
DS-QA-004 — Regression Testing and Determinism Suites	10
DS-QA-005 — Deterministic Financial Calculation Test Requirements	10
6. Safety-Boundary Testing	10
DS-QA-006 — Risk Engine Authority and Bypass-Resistance Testing	10
DS-QA-007 — Sage Advisory Boundary Testing	11
DS-QA-008 — AI Output Validation Testing	11
DS-QA-009 — TradeValidationPipeline Stage-Order Testing	11
7. Backtesting and Model Evaluation	12
DS-QA-010 — Backtesting Validation	12
DS-QA-011 — Model and AI Evaluation	13
8. Data Quality and Performance	13
DS-QA-012 — Data Quality and Freshness Testing	13
DS-QA-013 — Security Testing	13
DS-QA-014 — Performance and Responsiveness Testing	14
DS-QA-015 — Accessibility Testing	14
9. End-to-End Testing and Release Gates	14
DS-QA-016 — End-to-End Workflow Testing	14
DS-QA-017 — Release Gates	15
10. Traceability and Defect Management	15
DS-QA-018 — Requirement-to-Test Traceability	15
DS-QA-019 — Defect Severity Classification	16

DS-QA-020 — Test Data and Fixture Management	16
DS-QA-021 — Interface State Lifecycle Testing	17
DS-QA-022 — Audit-Log Integrity Testing	17
DS-QA-023 — Incident-Response Lifecycle Testing	18
10a. Research, Journal, and Trade Intelligence Testing (added in the independent-audit H1 repair)	19
DS-QA-024 — Research Intelligence Testing	19
DS-QA-025 — Journal and Review Testing	19
DS-QA-026 — Trade Intelligence Package Testing	19
DS-QA-027 — Trade Intelligence Package Security Testing (added for independent-audit Blocker 2)	
DS-QA-028 — Journal Retention and Deletion Testing (added for independent-audit Blocker 3) .	20
11. Non-Goals	21
12. Dependencies	21
13. Risks and Constraints	21
14. Verification Approach	21
15. References	22
Appendix A — Open Questions	22
16. Founder Vision Completion Test Program	22

1. Purpose

DS-009 is the authoritative testing and quality-assurance specification for DarkSage. Every requirement across DS-002 through DS-008 already states its own Testing field; DS-009 does not restate those individually. Instead, it defines the test *categories*, the *release-gate* mechanism, the *traceability* obligation that ties requirements to tests, and the *defect-severity* vocabulary this Codex's own audit process already uses informally — making that vocabulary formal and consistent for product code as well as documentation.

2. Scope

This document governs:

- test category definitions (unit, integration, contract, regression, end-to-end, security, performance, data-quality, model evaluation, backtesting validation, deterministic financial calculation);
- release gates tied to ROADMAP.md phase exit criteria and the live-trading gate (DS-EXE-007);
- requirement-to-test traceability;
- defect severity classification; and
- test data/fixture management principles.

DS-009 does not govern: product-level feature commitments (DS-002), technical architecture (DS-004), database schema (DS-005), API contracts (DS-006 — authoritative for what a contract test verifies against), security controls (DS-008 — authoritative for what a security test verifies against), UI presentation (DS-007), specific test framework/tooling selection, or CI/CD pipeline configuration (DS-010 concern). DS-009 tests against the other volumes' already-approved obligations; it does not create new product requirements.

3. Audience

Engineering contributors, QA contributors, independent auditors, and future Codex authors defining test suites for new requirements.

4. Definitions

See DS-001 §24 and DS-002 §4. Additional terms:

Term	Meaning
Release gate	A documented set of test/verification results that must pass before a phase, feature, or trading-mode transition is considered complete
Traceability	The recorded link from a requirement (any DS-<PREFIX>-NNN ID) to the test(s) that verify it, per docs/traceability/TRACEABILITY_MATRIX.csv
Fixture	A version-controlled, deterministic input/output pair used to test a calculation or behavior reproducibly

Term	Meaning
Defect severity	The Critical/High/Medium/Low classification of a defect's impact, independent of the Release Classification of the requirement it violates

5. Test Category Definitions

DS-QA-001 — Requirement Test-Coverage Completeness Baseline

Release Classification: Committed / MVP | **Governing Source:** DS-NFR-006 (Committed)

Description: This requirement restates DS-NFR-006 as a testing-architecture obligation rather than redefining it: every Committed/MVP requirement across DS-002 through DS-008 has at least one associated automated or documented manual test, recorded in that requirement's own Testing field. No Committed/MVP requirement in this Codex has an empty or missing Testing field.

Acceptance Criteria: Matches DS-NFR-006's acceptance criteria exactly, extended to cover DS-006/DS-007/DS-008 (which did not exist when DS-NFR-006 was authored).

Testing: Requirements-completeness audit confirming every Committed/MVP requirement across DS-002 through DS-009 has a non-empty Testing field.

DS-QA-002 — Integration Testing

Release Classification: Committed / MVP | **Governing Source:** DS-ARC-019 (Committed)

Description: Integration tests verify that DS-ARC-019's no-tight-coupling boundaries actually hold in practice, not only in module-dependency inspection: each adapter boundary (market-data provider, chart engine, AI provider, broker adapter, once each exists) is exercised through its real interface with a fixture implementation on the other side.

Acceptance Criteria: A second/fixture implementation can be substituted at each adapter boundary in a test environment without modifying the consuming feature's code (shared with DS-ARC-006/007/008/013/019's own tests).

Testing: Provider-substitution integration test per adapter boundary (shared with the relevant DS-ARC requirement's own test).

DS-QA-003 — API Contract Testing

Release Classification: Committed / MVP | **Governing Source:** DS-006 (Committed, authoritative for callable behavior)

Description: DS-006 is the authoritative source of truth for every callable client/backend contract; DS-009 does not re-derive that contract, only verifies conformance to it. Contract tests verify, per endpoint group, that the implementation matches DS-006's stated method, path, path/query parameters, request/response body shape, success status, and error semantics — including the deterministic collection/item operation tables DS-006's H4 repair established (Scan Configuration, Integration Credentials, Alerts, Notifications, candle timeframe/range/filtering) and the session lifecycle contract (DS-API-COR-010).

Acceptance Criteria:

- No implementation ships an endpoint whose behavior diverges from its DS-006 contract without DS-006 being updated first (contract-first, not implementation-first).
- A contract test exists for every Committed/MVP DS-API-<DOMAIN>-NNN requirement.

Testing: Automated contract-conformance test suite per endpoint group, generated or maintained against DS-006's stated schemas.

DS-QA-004 — Regression Testing and Determinism Suites

Release Classification: Committed / MVP | **Governing Source:** DS-PRD-004 (Committed)

Description: This requirement restates DS-PRD-004's acceptance criteria as a testing-architecture obligation: given fixed input fixtures, a material financial calculation reproduces identical output byte-for-byte (or within a defined floating-point tolerance) across repeated runs and across code versions, unless the calculation itself changed — in which case the change is recorded (Revision History or an equivalent changelog entry), not silently absorbed into a "new baseline."

Acceptance Criteria: Matches DS-PRD-004's acceptance criteria exactly.

Testing: Deterministic-output regression suite run on every change to calculation code, comparing against version-controlled fixture expectations (DS-QA-020).

DS-QA-005 — Deterministic Financial Calculation Test Requirements

Release Classification: Committed / MVP | **Governing Source:** ADR-003; DS-PRD-004, DS-RSK-002 (both Committed)

Description: Every material financial calculation — position value, P/L, risk metric (DS-RSK-002), backtest statistic (DS-BKT-001, once implemented) — has a dedicated deterministic regression suite against fixture data before it ships. No material financial calculation is considered Committed/MVP-complete without one.

Acceptance Criteria:

- No material financial calculation merges without an accompanying fixture-based regression test.
- Each Committed/MVP calculation's documented method (per DS-RSK-002's acceptance criteria) is referenced from its test, not only its output.

Testing: Deterministic-output regression suite against fixture positions/market data (shared with DS-RSK-002's own test).

6. Safety-Boundary Testing

DS-QA-006 — Risk Engine Authority and Bypass-Resistance Testing

Release Classification: Committed / MVP | **Governing Source:** DS-PRD-006, DS-RSK-001 (both Committed); DS-API-RSK-001 (DS-006, Committed)

Description: Adversarial tests attempt to have any caller — a Sage-issued instruction, a direct API call to GET /risk/state or any other endpoint (DS-API-RSK-001), or a narrated workaround — bypass, suppress, or override a Risk Engine determination. Every attempt is confirmed to fail and to be logged. Risk Engine unavailability is tested to confirm fail-safe (block/warn) behavior, never an implicit allow.

Acceptance Criteria: Matches DS-PRD-006/DS-RSK-001's acceptance criteria exactly, extended to explicitly include the DS-API-RSK-001 read-only API boundary.

Testing: Adversarial bypass-attempt test (shared with DS-PRD-006/DS-RSK-001's own test); fail-safe-on-unavailability test.

DS-QA-007 — Sage Advisory Boundary Testing

Release Classification: Committed / MVP (governance boundary); Planned (full execution, Phase 6) | **Governing Source:** DS-PRD-005 (Committed); DS-003 Core Rule, DS-SGE-005 (Planned)

Description: Adversarial tests confirm Sage can never directly submit an order or access/call the Execution Engine or Broker Adapter, under any circumstance, including after user confirmation (DS-003 Core Rule). UI/behavioral tests confirm recommendation language is visually and textually distinguishable from confirmed-action language (DS-PRD-005, DS-UX-021) and that no consequential action completes without a distinct, attributable user confirmation event separate from Sage's own output.

Acceptance Criteria: Matches DS-PRD-005/DS-003 Core Rule's acceptance criteria exactly; the governance-boundary test (no direct execution access) is testable now as a code-boundary/static-analysis check even before Sage is built, mirroring DS-EXE-001's own pre-implementation testability.

Testing: Adversarial execution-access test (shared with DS-003 Core Rule's own test, testable now as a static boundary check); UI/behavioral confirmation-distinction test (Phase 6, shared with DS-PRD-005's own test).

DS-QA-008 — AI Output Validation Testing

Release Classification: Committed / MVP | **Governing Source:** DS-PRD-011, DS-SCA-008 (both Committed)

Description: Adversarial tests craft AI output attempting a disallowed action (e.g., embedded shell-command-like text, a fabricated "override risk controls" instruction) and confirm it is rejected before reaching any sensitive code path, per DS-PRD-011/DS-SCA-008.

Acceptance Criteria: Matches DS-PRD-011's acceptance criteria exactly.

Testing: Adversarial AI-output-injection test (shared with DS-PRD-011/DS-SCA-008's own test).

DS-QA-009 — TradeValidationPipeline Stage-Order Testing

Release Classification: Committed / MVP for the unconditional pipeline-integrity constraint; Planned for the conditional Trade Proposal submission test | **Governing Source:** DS-EXE-001,

DS-API-EXE-001 (both Committed governance boundaries); DS-API-EXE-005, DS-API-EXE-006 (both Planned, DS-006 §6.14 — never treated as Committed by this requirement)

Description: This requirement has two distinct, separately classified parts, corrected in the DS-009-H1 repair to stop conflating them. **(1) Always-required pipeline-integrity constraint (Committed/MVP):** tests confirm no code path, present or future, submits an order or reaches the Broker Adapter without passing every canonical TradeValidationPipeline stage (ARCHITECTURE.md §14) in full and in order — Signal Validator → Strategy Validation → Risk Engine → Permissions Engine → Portfolio/Exposure → Buying Power → Market Condition → Order Validation → Execution Engine → Broker Adapter — per DS-EXE-001/DS-API-EXE-001. This part is testable now, as a requirements-review/static-analysis check, and binds regardless of which endpoints exist. **(2) Conditional Trade Proposal submission testing (Planned):** once DS-API-EXE-005 (Trade Proposal creation) and DS-API-EXE-006 (Trade Proposal submission into the pipeline) are implemented — both Planned, Phase 7 — tests additionally confirm creation alone never reaches execution and that submission cannot skip, reorder, or short-circuit a stage. This conditional part is not testable and does not apply until DS-API-EXE-006 itself is built; its Planned classification is unaffected by part (1)'s Committed status, and DS-API-EXE-006 is never promoted to Committed by virtue of this requirement.

Acceptance Criteria:

- Part (1) (always-required): matches DS-EXE-001/DS-API-EXE-001's acceptance criteria exactly, testable now via requirements review.
- Part (2) (conditional, Planned): once DS-API-EXE-005/006 exist, no implementation may skip, reorder, or short-circuit a pipeline stage from the Trade Proposal submission path; this criterion is not evaluated, and does not block any release gate, before those endpoints exist.
- No audit or test report may cite this requirement's Committed classification as evidence that DS-API-EXE-005 or DS-API-EXE-006 is Committed.

Testing: Requirements review now, covering part (1) only (shared with DS-EXE-001's own test) — always required, regardless of implementation phase. Full pipeline-stage-order adversarial test covering part (2), deferred until DS-API-EXE-005/006 are implemented (Phase 7) — conditional, not required before then.

7. Backtesting and Model Evaluation

DS-QA-010 — Backtesting Validation

Release Classification: Planned, Phase 2 | **Governing Source:** DS-BKT-001, DS-BKT-002, DS-BKT-003, DS-BKT-004 (all Planned)

Description: Backtesting validation tests cover: deterministic reproducibility of backtest results from identical inputs (DS-BKT-001); realistic transaction-cost/slippage modeling (DS-BKT-002); look-ahead-bias and data-leakage prevention, confirming a backtest never uses information unavailable at the simulated point in time (DS-BKT-003); and presence of the required disclosure that historical/simulated performance is not a guarantee of future performance (DS-BKT-004, also required in the API response per DS-API-BKT-001).

Acceptance Criteria: Matches DS-BKT-001 through 004's acceptance criteria exactly.

Testing: Deferred to DS-BKT's own implementation timing (Phase 2); disclosure-presence is testable as a response-schema contract now (shared with DS-API-BKT-001's own test).

DS-QA-011 — Model and AI Evaluation

Release Classification: Planned, Phase 6 | **Governing Source:** DS-SGE family (Planned); DS-PRD-002, DS-PRD-009 (both Committed)

Description: Once Sage exists, model/AI evaluation tests assess output quality against evidence-grounding (DS-PRD-002 — does a response cite the evidence it claims to rely on), confidence-calibration (DS-PRD-009 — is uncertainty communicated consistently and honestly), and non-fabrication (does the response avoid presenting inference as fact, DS-SGE-001). Model evaluation is explicitly never a substitute for the deterministic-calculation testing DS-QA-005 requires — a model's self-reported confidence or a favorable evaluation score never authorizes treating generative output as a financial calculation (DS-PRD-004).

Acceptance Criteria:

- No model-evaluation result is used to justify skipping or weakening a deterministic-calculation test elsewhere in this document.
- Evidence-grounding, confidence-calibration, and non-fabrication are each independently scored, not collapsed into one opaque "quality" metric.

Testing: Deferred to DS-AI/DS-SGE's own implementation timing (Phase 6); the non-substitution constraint is testable now as a review-time requirement (does any planned test conflate model evaluation with deterministic verification).

8. Data Quality and Performance

DS-QA-012 — Data Quality and Freshness Testing

Release Classification: Committed / MVP | **Governing Source:** DS-PRD-008, DS-MKT-004, DS-DAT-003 (all Committed)

Description: Feed-interruption simulation tests confirm stale/delayed indicators appear within DS-MKT-004's defined threshold window (10 seconds real-time, delay+60 seconds delayed, 1 hour post-close end-of-day). Symbol-identity continuity and ticker-reuse-disambiguation tests confirm DS-DAT-003's canonical-identity guarantee holds across a ticker change or a delisting-and-reuse scenario.

Acceptance Criteria: Matches DS-PRD-008/DS-MKT-004/DS-DAT-003's acceptance criteria exactly.

Testing: Feed-interruption simulation test; ticker-change continuity and ticker-reuse disambiguation regression test (shared with the respective requirements' own tests).

DS-QA-013 — Security Testing

Release Classification: Committed / MVP | **Governing Source:** SECURITY_RULES.md "Security Testing"; DS-008 (Committed)

Description: Security testing includes authentication tests, authorization tests, secret scanning, dependency scanning, input-validation tests, API-abuse tests, rate-limit tests, broker-safety tests, and fault injection — the verbatim category list SECURITY_RULES.md states. Fault-injection scenarios include broker disconnect, data-provider disconnect, database failure, network timeout, duplicate requests, partial broker responses, backend restart, mobile disconnect, and desktop

crash. This document does not redefine the controls being tested — DS-008 is authoritative for those.

Acceptance Criteria:

- Every listed security-test category has a corresponding automated or documented test once its underlying feature exists; Committed/MVP security controls (DS-SCA-001 through 011, 015, 018, 020, 021, 023, 024) are tested now.
- Every row of DS-SCA-024's material threat-scenario table has its Required Control mapped to at least one test in this document or docs/traceability/TRACEABILITY_MATRIX.csv, per DS-SCA-024's own acceptance criteria — this is the specific test-alignment mechanism for the actionable threat model added in the DS-008-H1 repair.

Testing: Security test suite execution per category, shared with each corresponding DS-SCA-NNN requirement's own Testing field; scenario-to-test traceability audit against DS-SCA-024's table (shared with DS-SCA-024's own test).

DS-QA-014 — Performance and Responsiveness Testing

Release Classification: Committed / MVP for startup; Planned for interaction-latency |

Governing Source: DS-NFR-001 (Committed), DS-NFR-002 (Planned)

Description: An automated startup-time regression benchmark verifies DS-NFR-001's cold-start (10s) and warm-start (3s) budgets on the defined reference hardware. Once DS-NFR-002 is implemented, an automated latency regression benchmark verifies each Committed/MVP interactive operation (widget add/remove/resize, layout drag, watchlist edit) against its documented maximum-latency budget, separating local UI responsiveness from external data latency where an operation depends on both.

Acceptance Criteria: Matches DS-NFR-001/002's acceptance criteria exactly.

Testing: Automated startup-time regression benchmark (shared with DS-NFR-001's own test); automated interaction-latency regression benchmark once DS-NFR-002 is implemented.

DS-QA-015 — Accessibility Testing

Release Classification: Planned | **Governing Source:** DS-NFR-004, DS-UX-017 (both Planned)

Description: Keyboard-only workflow completion tests confirm every Committed/MVP workflow, including workspace layout editing, is completable without a pointing device. An automated accessibility-conformance scan verifies the documented standard (e.g., WCAG 2.2 Level AA) once adopted.

Acceptance Criteria: Matches DS-NFR-004/DS-UX-017's acceptance criteria exactly.

Testing: Keyboard-only workflow completion test; automated accessibility-conformance scan (shared with DS-NFR-004/DS-UX-017's own tests).

9. End-to-End Testing and Release Gates

DS-QA-016 — End-to-End Workflow Testing

Release Classification: Committed / MVP | **Governing Source:** ROADMAP.md phase exit criteria

Description: Golden-path end-to-end tests verify each phase's ROADMAP.md exit criteria directly (e.g., Phase 1: "desktop app launches," "backend starts locally," "indicators match reference tests"). A phase is not recorded complete in .ai-workflow/release documentation until its exit-criteria E2E tests pass.

Acceptance Criteria:

- Every ROADMAP.md phase exit criterion has a corresponding E2E test or documented manual verification procedure.
- No phase is marked complete in project tracking without its E2E verification recorded.

Testing: Phase-exit-criterion E2E test suite, one per ROADMAP.md phase.

DS-QA-017 — Release Gates

Release Classification: Committed / MVP | **Governing Source:** ROADMAP.md phase exit criteria; DS-EXE-007 (Committed)

Description: A phase's ROADMAP.md exit criteria and this document's applicable test categories (DS-QA-001 through 016, as relevant to that phase's scope) must pass before the phase is recorded complete. The live-trading release gate is DS-EXE-007's prerequisite list (paper performance acceptable, independent security review passed, broker reconciliation passed, kill switch tested, data-health checks passed, duplicate-order prevention tested, monitoring active, explicit separate user unlock) verified through the corresponding DS-QA test categories (DS-QA-006/009/010/013) — DS-009 does not redefine DS-EXE-007's prerequisite list, only maps it to the tests that verify it.

Acceptance Criteria:

- No phase completion is recorded without its applicable release-gate tests passing.
- Live trading is never enabled without every DS-EXE-007 prerequisite's corresponding test passing.

Testing: Release-gate checklist execution per phase; live-trading gate checklist execution (shared with DS-EXE-007's own test).

10. Traceability and Defect Management

DS-QA-018 — Requirement-to-Test Traceability

Release Classification: Committed / MVP | **Governing Source:** DS-002 §5.5 (five-stage traceability model)

Description: Every Committed/MVP requirement across DS-002 through DS-009 maps to at least one recorded test in docs/traceability/TRACEABILITY_MATRIX.csv, extending DS-002 §5.5's five-stage model (Requirement → Design/ADR → Source → Test → Release/Change) to now include DS-006, DS-007, DS-008, and DS-009 as requirement sources, not only DS-002's own families.

Acceptance Criteria:

- Every Committed/MVP DS-<PREFIX>-NNN ID across all volumes has a traceability-matrix row.
- A requirement's Test/Release fields move from Pending to populated as its test/release evidence becomes available, never silently left Pending after the test exists.

Testing: Traceability-completeness audit: every Committed/MVP requirement ID across DS-002 through DS-009 appears in the matrix with a non-empty Test link once its test exists.

DS-QA-019 — Defect Severity Classification

Release Classification: Committed / MVP | **Governing Source:** DS-009 (this document, normative); DS-DEV-009, DS-DEV-025 (DS-010, both Committed); DS-ADR-010 (DS-012, Committed)

Description: Defects are classified Critical, High, Medium, or Low, independent of the Release Classification of the requirement they violate: **Critical** — a safety, financial-correctness, security, privacy, reproducibility, or architecture-boundary breach, or a Risk Engine/TradeValidationPipeline bypass; **High** — a violation of a Committed/MVP requirement's acceptance criteria, or a Committed-requirement-depends-on-Planned-capability defect; **Medium** — a violation of a Planned requirement's acceptance criteria, or a non-safety-critical UX defect; **Low** — cosmetic or non-blocking. This severity vocabulary is normative on DS-009's own authority and is the single definition applicable to both product code defects and this Codex's own documentation-audit findings — it is not derived from, and does not depend on, any local workflow file. DS-DEV-009 (DS-010, Committed) references this scale as the basis for its Critical/High blocker-handling rule, and DS-ADR-010 (DS-012, Committed) places local workflow files at the lowest tier of its conflict-resolution hierarchy, never authoritative for this or any other Committed/MVP requirement (DS-DEV-025). A local, non-committed file such as `.ai-workflow/BLOCKERS.md` may separately record defects using this vocabulary for day-to-day coordination; that record is historical/operational context only, is never required reading to define or apply this severity model, and never varies or supersedes the definition stated here.

Severity Precedence Rule: A single defect may match the stated criteria for more than one severity level at once (e.g., it both breaches security and violates a Committed/MVP acceptance criterion). When this happens, the **highest-matching severity governs** — Critical outranks High, High outranks Medium, Medium outranks Low — and the defect is recorded under that single highest severity, never split across multiple severity records or averaged down. This precedence order exists specifically so that Critical/High findings touching correctness, security, safety, privacy, reproducibility, or architecture can never be reclassified downward by also matching a lower-severity criterion (e.g., a security breach that happens to also look like a "cosmetic" UX defect remains Critical, not Low). No existing blocker category is downgraded by this rule; it only makes explicit, deterministic, and auditable a resolution order that the four bullet definitions already implied but did not state.

Acceptance Criteria:

- Every recorded defect carries exactly one severity per this scale, resolved via the Severity Precedence Rule when more than one level's criteria are met.
- A Critical defect blocks release of the affected capability; a High defect blocks release of the affected Committed/MVP requirement; Medium/Low defects do not block release by themselves.
- No defect matching a Critical or High criterion is ever recorded at a lower severity because it also matches a Medium or Low criterion.

Testing: Defect-triage process audit confirming severity assignment consistency against this scale's definitions.

DS-QA-020 — Test Data and Fixture Management

Release Classification: Committed / MVP | **Governing Source:** DS-QA-004, DS-QA-005 (both Committed, this document)

Description: Deterministic fixtures for financial calculations, market data, and API contract tests are version-controlled and reused across unit, regression, and backtest tests, preventing fixture drift from silently changing an "expected" result without a recorded, reviewed change.

Acceptance Criteria:

- A change to a shared fixture's expected output is a reviewed, recorded change (not a silent update to make a failing test pass).
- Fixtures used by more than one test category (e.g., a position fixture used by both DS-QA-005 and DS-QA-006) are defined once and referenced, not duplicated with drift risk.

Testing: Fixture-drift audit: confirm no fixture expectation changed without a corresponding recorded change entry.

DS-QA-021 — Interface State Lifecycle Testing

Release Classification: Committed / MVP for the core lifecycle; Planned for assistive-technology announcement testing | **Governing Source:** DS-UX-016 (DS-007, Committed); DS-UX-022 (DS-007, Planned)

Description: Added in the cross-volume alignment portion of the DS-007/008/009 repair pass to make DS-UX-016's Interface State Lifecycle testable, per DS-QA-001's baseline obligation that every Committed/MVP requirement has an associated test. **Core lifecycle testing (Committed/MVP):** a state-transition regression suite exercises every transition DS-UX-016 defines (Loading→Ready, Loading→Error, Ready→Refreshing, Refreshing→Ready, Refreshing→Degraded/Partial, Degraded/Partial→Ready, any state→Error, Error→Retry/Recovery, Retry/Recovery→Loading or →Refreshing) for each Committed/MVP data-bearing surface, confirming previously valid data is preserved where safe during Refreshing and that Degraded/Partial states disclose which portion of expected data is affected. **Accessibility announcement testing (Planned):** once DS-UX-022 is implemented (itself tied to DS-NFR-004/DS-UX-017), an assistive-technology announcement test confirms each state entry is announced. Corrected in the DS-009-H1 narrow repair: this conditional Planned test is not a Committed/MVP release-gate requirement and is never evaluated as blocking before DS-UX-022 exists.

Acceptance Criteria:

- Core lifecycle (Committed/MVP): matches DS-UX-016's acceptance criteria exactly.
- Accessibility announcements (Planned): matches DS-UX-022's acceptance criteria exactly, once implemented; not evaluated before then.

Testing: State-transition regression test per Committed/MVP data-bearing surface (shared with DS-UX-016's own test) — always required. Assistive-technology state-announcement test (shared with DS-UX-022's own test) — conditional, deferred until DS-UX-022 is implemented.

DS-QA-022 — Audit-Log Integrity Testing

Release Classification: Committed / MVP | **Governing Source:** DS-SCA-025, DS-SCA-027 (both DS-008, Committed)

Description: Added in the cross-volume alignment portion of the DS-007/008/009 repair pass to make DS-SCA-025's audit-log trustworthiness architecture testable; expanded in the DS-009-H2

narrow repair to cover DS-SCA-027's integrity-verification contract, going beyond confirming that normal API-level modification attempts are rejected. **Access/write-boundary testing (DS-SCA-025):** a tamper-attempt adversarial test exercises every available code path in an attempt to modify or delete an existing audit record via the application's own API/write path and confirms rejection; an unauthorized-read rejection test confirms a permission group outside DS-SCA-025's authorized-reader set cannot read the audit log; a retention-boundary audit confirms documented retention/archival boundaries are distinct from general application log retention. **Integrity-verification testing (DS-SCA-027):** a direct-storage-corruption adversarial test bypasses the application's API entirely and directly manipulates the underlying storage to (a) modify an existing record's content, (b) reorder records, (c) truncate the sequence, (d) selectively delete a record, and (e) insert an unauthorized record — each confirmed detected at the next verification trigger, not merely rejected at write time. Verification-trigger tests confirm verification runs at startup, before a security-sensitive audit review, on the scheduled/background cadence, and immediately after a simulated storage/persistence fault. A fail-closed test confirms a security-sensitive or trading-relevant operation is blocked while a Violation Detected or Verification Inconclusive state is unresolved. An evidence-preservation test confirms the compromised range is preserved unmodified during a simulated investigation, and an anti-silent-rebuild test confirms no code path regenerates or rewrites the compromised range to make verification pass again without first escalating per DS-SCA-026.

Acceptance Criteria:

- Access/write-boundary portion: matches DS-SCA-025's acceptance criteria exactly.
- Integrity-verification portion: matches DS-SCA-027's acceptance criteria exactly — every detection category (modification, reordering, truncation, selective deletion, unauthorized insertion) is demonstrated via direct storage manipulation, not only an API-level attempt; every verification trigger is exercised; no test accepts a silently-rebuilt or silently-resolved violation as passing.

Testing: Tamper-attempt adversarial test (API-level); unauthorized-read rejection test; fail-closed-on-audit-unavailability test; retention-boundary audit (all shared with DS-SCA-025's own test). Direct-storage-corruption adversarial test per detection category; startup/pre-review/scheduled/post-fault verification-trigger tests; fail-closed-on-violation test; evidence-preservation test; anti-silent-rebuild test (all shared with DS-SCA-027's own test).

DS-QA-023 — Incident-Response Lifecycle Testing

Release Classification: Committed / MVP | **Governing Source:** DS-SCA-026 (DS-008, Committed)

Description: Added in the cross-volume alignment portion of this repair pass to make DS-SCA-026's security-incident response lifecycle testable. A tabletop/simulated-incident walkthrough is performed per incident type DS-SCA-026 lists (compromised session, compromised broker credentials, suspected unauthorized trade activity, compromised update/dependency path, audit-log integrity failure, malicious/corrupted external data or integration, local-device compromise where detectable), confirming each applicable lifecycle step (Detection through Post-Incident Review) occurs in order. A live-trading re-enablement test confirms the Validation-Before-Re-Enabling step cannot be bypassed for a live-trading-relevant incident regardless of severity classification.

Acceptance Criteria: Matches DS-SCA-026's acceptance criteria exactly.

Testing: Tabletop/simulated-incident walkthrough per listed incident type; live-trading re-enablement gate-bypass adversarial test (both shared with DS-SCA-026's own test).

10a. Research, Journal, and Trade Intelligence Testing (added in the independent-audit H1 repair)

DS-QA-024 — Research Intelligence Testing

Release Classification: Planned | **Governing Source:** DS-RSH-001 through DS-RSH-006 (DS-002, Planned); DS-ARC-026 (DS-004, Planned); DS-API-RSH-001/002/003 (DS-006, Planned)

Description: Testing for Research Intelligence covers: evidence-completeness (every ResearchEvidence record carries source/publication-time/retrieval-time), per-domain adapter substitution/disable regression, conflicting-evidence disclosure (DS-RSH-005), catalyst-timeline ordering and date-uncertainty disclosure (DS-RSH-003), and thesis-immutability/revision-chain integrity (DS-RSH-004).

Acceptance Criteria: Matches DS-ARC-026/DS-API-RSH-001..003's acceptance criteria exactly.

Testing: Adapter-substitution test per research domain; evidence-immutability audit; thesis-revision-chain integrity test; conflicting-evidence disclosure test (all shared with DS-ARC-026/DS-DB-029/030/031's own tests).

DS-QA-025 — Journal and Review Testing

Release Classification: Planned | **Governing Source:** DS-JRN-001 through DS-JRN-005 (DS-002, Planned); DS-JRN-006 (DS-002, Future/Exploratory — not in scope of this testing requirement); DS-ARC-027 (DS-004, Planned); DS-API-JRN-001/002/003 (DS-006, Planned)

Description: Testing for Journal & Review Intelligence covers: original-plan immutability and amendment-attribution (DS-JRN-002), deterministic-figure-reuse in daily/weekly reviews (confirming no review statistic is independently computed outside DS-PERF/DS-PRT), correlation-vs-causation content review for weekly pattern-discovery notes (DS-JRN-004), and the Sage coaching non-diagnosis boundary (DS-JRN-005).

Acceptance Criteria: Matches DS-ARC-027/DS-API-JRN-001..003's acceptance criteria exactly.

Testing: Original-plan immutability and amendment-attribution test; deterministic-figure-reuse audit; non-diagnosis boundary test (all shared with DS-ARC-027/DS-DB-032/033's own tests).

DS-QA-026 — Trade Intelligence Package Testing

Release Classification: Planned | **Governing Source:** DS-SIG-005 (DS-002, Planned); DS-ARC-028 (DS-004, Planned); DS-API-TIP-001 (DS-006, Planned)

Description: Testing for the canonical Trade Intelligence Package covers: cross-surface consistency (chart, journal, alert, and audit renderings of the same package_id/version show identical field values), field-provenance (every deterministic field traces to its owning engine, never to generated text), and null-vs-fabricated-field handling (an unavailable field is explicit/null, never a

plausible-looking substitute).

Acceptance Criteria: Matches DS-ARC-028/DS-API-TIP-001's acceptance criteria exactly.

Testing: Cross-surface consistency test; field-provenance audit; null-vs-fabricated-field regression test (all shared with DS-ARC-028/DS-DB-028's own tests).

DS-QA-027 — Trade Intelligence Package Security Testing (added for independent-audit Blocker 2)

Release Classification: Planned | **Governing Source:** DS-SCA-029 (DS-008, Planned)

Description: Testing for DS-SCA-029's Trade Intelligence Package security authority covers: field-immutability-once-referenced (no in-place edit to a package version any downstream consumer has already read); provenance-retrieval (every field's owning-service attribution is present and correct); direct-storage tamper-detection (bypassing the API to modify/reorder/insert a package version is detected at the next verification trigger, sharing DS-SCA-027's mechanism); AI-deterministic-override adversarial testing (attempting to have Sage or any provider write a deterministic TIP field without an auditable engine call, and confirming rejection); stale/partial-disclosure (a package with expired freshness or partial assembly is never presented as complete/current); external-redaction (capital-at-risk/account-context fields never leave the product beyond explicit configuration, shared with DS-ALT-004's own test); audit-correlation (every consequential TIP change produces a matching AuditLogEntry); and fail-closed-on-unverifiable-package (a consequential action depending on an integrity- or freshness-unverifiable package is blocked).

Acceptance Criteria: Matches DS-SCA-029's acceptance criteria exactly.

Testing: Field-immutability-once-referenced regression test; provenance-retrieval audit; direct-storage tamper-detection test (shared with DS-SCA-027's own test); AI-deterministic-override adversarial test (shared pattern with DS-SGE-003/DS-PRD-011's own tests); stale/partial-disclosure test; external-redaction test; audit-correlation test; fail-closed-on-unverifiable-package test (all shared with DS-SCA-029's own test).

DS-QA-028 — Journal Retention and Deletion Testing (added for independent-audit Blocker 3)

Release Classification: Planned | **Governing Source:** DS-JRN-007 (DS-002, Planned); DS-API-JRN-004 (DS-006, Planned); DS-SCA-028 (DS-008, Planned)

Description: Testing for journal retention and deletion covers: deletion-propagation (a deleted JournalEntry's private content is removed along with its attachment references and any derived DailyReview/WeeklyReview narrative or Sage-drafted commentary that would reproduce it, while deterministic performance figures in those reviews remain intact); immutable-record-preservation (a deleted JournalEntry's linked Transaction/AuditLogEntry references are never themselves deleted, and a legal/audit-exception-retained record discloses exactly which factual fields were kept); backup behavior (deleted content is purged from backups on a disclosed schedule, or the residual-retention window is disclosed to the user, so a backup restore never silently reverses a deletion without disclosure); attachment handling (chart-snapshot/attachment references are deleted alongside their parent entry); export-before-delete (an export is available prior to deletion where DS-DAT-002 export is supported); deletion-failure disclosure (a failed deletion reports its reason rather than appearing to silently succeed); the conflict case (deletion against an entry with

an active conflicting reference, e.g., an unresolved TradingThesis link, returns a disclosed conflict rather than silently orphaning the reference); and privacy-safe logging (no deletion-related audit-log entry reproduces the deleted private content — identifier, timestamp, and outcome only).

Acceptance Criteria: Matches DS-JRN-007/DS-API-JRN-004's acceptance criteria exactly.

Testing: Deletion-propagation regression test; immutable-record-preservation test; secure-deletion/backup-disclosure test (shared with DS-SCA-028's own test); attachment-deletion test; export-before-delete test; deletion-failure disclosure test; conflict (409) regression test; privacy-safe-logging audit (all shared with DS-JRN-007/DS-API-JRN-004's own tests).

11. Non-Goals

DS-009 does not: select a specific test framework, runner, or CI/CD tooling (DS-010 concern); redefine any DS-006 API contract or DS-008 security control (it tests against them); commit new product capability via test authoring; or substitute model/AI evaluation (DS-QA-011) for deterministic-calculation verification (DS-QA-005) under any circumstance.

12. Dependencies

- DS-001, DS-002, DS-003, DS-004, DS-006, DS-007, DS-008
- DS-010 (DS-DEV-009, DS-DEV-025), DS-012 (DS-ADR-010)
- ROADMAP.md, SECURITY_RULES.md, TRADING_RULES.md
- docs/traceability/TRACEABILITY_MATRIX.csv

13. Risks and Constraints

- **Consolidation, not duplication:** most DS-QA-NNN requirements restate an already-Committed upstream acceptance criterion as a testing-architecture obligation rather than inventing new product behavior; each explicitly cites its Governing Source as "shared with [X]'s own test" to avoid two conflicting definitions of the same test existing in two volumes.
- **Classification discipline:** Planned test categories (DS-QA-010, 011, 015, and the Planned half of DS-QA-007/009/014/021) trace only to already-Planned upstream sources; no requirement here promotes a Planned upstream capability to Committed by virtue of being tested. DS-QA-021 was split in the DS-009-H1 narrow repair so its Committed core-lifecycle testing (DS-UX-016) no longer implies its Planned accessibility-announcement testing (DS-UX-022) is itself a Committed release gate.
- **Model evaluation boundary:** DS-QA-011 explicitly forbids model-evaluation results from substituting for deterministic-calculation tests, closing a plausible future failure mode (treating a "good" AI self-evaluation as equivalent to a passed deterministic regression suite) before it could occur.

14. Verification Approach

Each DS-QA-NNN requirement states its own Testing. Document-level verification (unique-ID check, cross-reference consistency against DS-002 through DS-008, no Committed requirement depending on a Planned-only capability) recorded in .ai-workflow/HANDOFF.md.

15. References

- ROADMAP.md, SECURITY_RULES.md, TRADING_RULES.md
- docs/codex/Volume-02-Product/DS-002-SRS.md and requirements/*.md
- docs/codex/Volume-04-Architecture/DS-004-Technical-Architecture.md
- docs/codex/Volume-06-API/DS-006-API-Specification.md
- docs/codex/Volume-07-UX/DS-007-UI-UX-Bible.md
- docs/codex/Volume-08-Security/DS-008-Security-Architecture.md
- docs/traceability/TRACEABILITY_MATRIX.csv

Appendix A — Open Questions

1. **Test framework/tooling selection** — explicitly out of scope for this document (§11); belongs to a future DS-010 addition.
2. **CI/CD pipeline configuration and gating automation** — the release-gate *content* is fixed here (DS-QA-017); the automation mechanism that enforces it in CI is a DS-010 concern not yet authored.
3. **Governance-confirmation carryover** — the standing BLOCKERS.md items (ROADMAP.md phase boundaries as Codex release-scope authority; phase-mapping precision) apply identically to this document's Release Classification scheme and are not re-litigated here.

16. Founder Vision Completion Test Program

Testing shall include: canonical Trade Intelligence Package schema/traceability; streaming-chart state and overlay consistency; agent tool-permission and prompt-injection red teams; research-source provenance/conflict/freshness; thesis-version immutability; journal hindsight protection; daily/weekly review reproducibility; automation-mode action matrices; emergency-stop and policy-expiry behavior; and portfolio/strategy deterministic calculations.

Discord tests shall cover opt-in defaults, secret redaction, test delivery, rate limits, duplicate suppression, retries, deleted webhook/permission loss, message-size handling, simulation/live labeling, in-app fallback, and proof that every Discord interaction path is unable to approve or execute a trade.